IIITDM KURNOOL / THIRD-YEAR CSE DBMS PROJECT

BusBook
Team User Manual

Run, share, use and demonstrate the bus ticket booking system

SYSTEM	BusBook · Oracle 21c XE + Django
AUDIENCE	Three-person project team
UPDATED	5 November 2026
SOURCE	Editable Markdown accompanies this PDF

Academic simulation. Bookings, payments, refunds, routes and fares are demonstration data; no real travel or payment service is provided.

IIITDM Kurnool · Third-year CSE DBMS project Updated: 5 November 2026 **Project:** Bus Ticket Booking System with Seat Reservation, Waiting List, and Cancellation Management

> This is an academic demonstration. Buses, routes, tickets, payments, refunds, charges and customer support are simulated. The app is hosted locally on one laptop; GitHub stores source and documents, not the running website or its Oracle data.

1. Choose how your team will use BusBook

Mode	Who can open it?	Internet while running?	What stays on the host?
One laptop	Only the host at http://127.0.0.1:8000/	No, after installation	Django and Oracle
Shared Wi-Fi or hotspot	Devices on the same local network, using the printed IP link	No; internet may also be present	Django and Oracle; each teammate uses a browser
Each person installs a copy	Each person runs an independent app	Needed for initial download, not normal local use	Separate Oracle data and website accounts on each machine
Public internet hosting	Not set up	Not applicable	Requires a separate production deployment

For the three-person demo, **one shared host is easiest**. The leader runs Oracle and the Django server on the Windows laptop. Teammates need only a browser and the same Wi-Fi or hotspot. A GitHub login, an Oracle database login, and a BusBook website login are three separate things.

2. Start the existing installation

1. Connect the host laptop to the intended Wi-Fi or hotspot. Confirm Oracle Database XE and its listener are running. If the laptop has just restarted, allow the xepdb1 service to register with the listener before opening the app.
2. Open PowerShell in the project folder and run:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\start_team.ps1
```

3. Enter the **Oracle BUSBOOKING password** when asked. This is not a passenger or Django admin password. The launcher refreshes the next 14 days of trips from recurring templates, starts the site, and prints the host and teammate links.
4. Keep that PowerShell window and the host laptop open. On the host, open http://127.0.0.1:8000/. Teammates join the same network and open the printed local IP link, for example http://192.168.1.10:8000/.
5. If you change Wi-Fi or hotspot, stop the server with **Ctrl+C**, rerun the launcher, and share its new link. Existing users and bookings stay in Oracle.

The -ExecutionPolicy Bypass option applies to this launched PowerShell process; it handles the Windows setting that blocked a direct ./start_team.ps1 call earlier. It does not make the site public. If Windows Firewall asks, allow Python on a trusted **Private** network. College Wi-Fi may prevent devices from talking directly; use a shared hotspot if the teammate link fails.

Run on the host laptop only

In a PowerShell window in the project folder, set the password and launch Django:

```
$secure = Read-Host "BUSBOOKING password" -AsSecureString
$env:DB_PASSWORD = [System.Net.NetworkCredential]::new("", $secure).Password
.\.venv\Scripts\python.exe manage.py runserver
```

DB_PASSWORD lasts only in that PowerShell window. Set it again in a new window before running database commands. Local booking still works if internet disappears, provided the host, Oracle and local network stay running. GitHub pushes wait until internet is available.

3. Set up a fresh Windows computer

Skip this section if you are only visiting the leader's shared app. A fresh installation starts with its own empty Oracle schema, no accounts and no fleet data.

Install Oracle Database 21c XE with the xepdb1 pluggable database, Python 3.14, a browser and Git. SQL Developer is recommended for DBMS demonstrations. Get the repository:

```
git clone https://github.com/karthik-goud227/bus-ticket-booking-system.git
cd bus-ticket-booking-system
```

In SQL Developer, connect as an Oracle administrator **to xepdb1**, choose a private password, and run the following only if the project account does not already exist:

```
CREATE USER BUSBOOKING IDENTIFIED BY "ChooseYourOwnPassword";
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE,
    CREATE VIEW, CREATE TRIGGER, CREATE PROCEDURE TO BUSBOOKING;
ALTER USER BUSBOOKING QUOTA UNLIMITED ON USERS;
```

Test a SQL Developer connection as BUSBOOKING on localhost:1521/xepdb1. In PowerShell:

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$secure = Read-Host "BUSBOOKING password" -AsSecureString
$env:DB_PASSWORD = [System.Net.NetworkCredential]::new("", $secure).Password
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py migrate
.\.venv\Scripts\python.exe manage.py createsuperuser
```

The last command creates a **website admin**, with its own username and password. migrate creates Oracle tables, keys, constraints, triggers, indexes and views; do not copy SQL from the report into the database to recreate them. For sample buses and trips, run seed_fleet, upgrade_fleet, then generate_daily_trips --days 14 in that order. These commands change persistent demo data and should not be used as a reset button on the leader's populated installation.

4. Create individual website accounts

Each person should open **Sign up** and choose a different passenger username and password. Sign in on separate browsers or devices for a realistic queue demonstration. The leader can sign in to /admin/ with the previously created Django superuser.

To give a trusted teammate an admin account, first let that teammate create a separate website username. The leader opens **Admin → Users**, selects the account, enables **Staff status**, and grants

the permissions needed for their role. **Superuser status** gives unrestricted control over users and project data; use it only if that full access is intended for a trusted teammate. Nobody needs to share the leader's password. The Oracle BUSBOOKING password is used by the host server, not by each passenger.

5. Prepare buses, trips and places

The existing shared installation has a varied demo fleet. A recorded verification on 2 November found 14 buses with at least 24 places each, 111 trips with complete inventory, three double-decker buses and 80 sleeping berths. Those were **dated results**, not guaranteed present-day counts. The launcher can add more future trips from daily templates; it does not erase bookings.

In /admin/, staff can maintain **Buses**, their physical **Seats** (standard or berth, upper or lower deck), **Recurring trips**, dated **Trips**, and timed **Trip points**. A bus may have many trips, but active time ranges for the same bus may not overlap. Trips on different buses can overlap. Each dated trip has its own TripSeat inventory, so booking a physical seat on one day does not book it on another day. A trip's departure and arrival determine its displayed approximate duration.

Daily recurring templates specify the bus, route, departure time, duration and fare. generate_daily_trips --days 14 creates a dated rolling window and skips duplicates or conflicting slots. It is not an infinite timetable. Staff should inspect the generated trips before a demo, especially after a date change. Staff can also add or edit boarding and drop-off **Trip points**. Every trip has default city bus-stand points; selected points and scheduled times appear on the ticket.

For a fresh installation only, the sequence below builds sample content. Keep DB_PASSWORD set in that PowerShell window:

```
.\.venv\Scripts\python.exe manage.py seed_fleet
.\.venv\Scripts\python.exe manage.py upgrade_fleet
.\.venv\Scripts\python.exe manage.py generate_daily_trips --days 14
.\.venv\Scripts\python.exe manage.py verify_fleet_layout
```

6. Passenger journey: search to ticket

1. **Search.** On the home page, use the From and To menus, travel date, bus type, operator, availability checkbox and departure/fare sort. The destination menu narrows when an origin is chosen. Results show operator, type, departure, approximate arrival and duration, fare per place, verified rating when available, and places available or full.
2. **Inspect the layout.** Open a trip. The map distinguishes available places (pink/red), selected places, and unavailable places (grey). It shows a steering wheel/front orientation, the exit, standard seats or sleeper berths, and upper/lower decks when that bus has them. Bus layouts may have three or four places per row. A full or closed trip cannot accept a direct hold.
3. **Choose details.** Select one or more available places. Enter a passenger name and phone number, and choose valid boarding and drop-off points. The selected boarding time must precede the drop-off time. The app no longer has the old six-seat limit; available inventory and the booking checks determine what can be selected.
4. **Read the bill.** Before committing, review the displayed total. The saved bill has base fare = trip fare × number of places, a ₹20 platform fee, a ₹10 service fee, and illustrative 18% GST on those two fees

(₹5.40). Thus a ₹499 trip fare for two places totals ₹1,033.40. The fee/tax treatment is part of this academic simulation, not a claim about any real operator's billing. Historical bookings made before this feature retain their original total.

5. **Hold.** Submit the selection. The places are held for **10 minutes**. The hold page shows the deadline and the detailed charge. Oracle transactions lock the relevant inventory so two simultaneous passengers cannot both reserve the same trip place. If another person got a place first, refresh the map and choose again.
6. **Confirm.** Use the **Confirm booking** action before the hold expires. It records a simulated captured payment and marks the places booked. No real money is charged.
7. **Ticket.** Open **My bookings → View ticket** to see the reference, passenger details, operator, route, deck/berth labels, date, boarding and drop-off times, bill and booking status. Use the browser's print action for a paper/PDF ticket. The printed ticket is a demo travel pass, not valid transport.

The app records the agreed bill and passenger details at booking time, so a later account-profile edit does not silently rewrite an old ticket. When a hold expires, its places are released when the relevant trip page or expiry command processes it; it is not an always-running background worker.

7. Cancellation and waiting list

From **My bookings**, open a booking and choose **Cancel booking** if available. The action cancels the entire booking, releases its places and, if payment was previously confirmed, records a simulated full refund. There is no partial cancellation. The booking, payment and status events remain for history. A trip that has departed cannot be cancelled through this flow.

For a full trip, a different signed-in passenger can choose **Join waiting list**. One active waiting entry per passenger per trip is allowed. **My bookings** shows the queue position. The queue is first-in, first-out by join time and row ID. When cancellation frees a place, the first waiting passenger receives a **single-place offer valid for 10 minutes**. On **My bookings**, that passenger selects journey points, provides passenger details, accepts the offer, then confirms the new held booking. Leaving the queue or letting an offer expire permits the next waiting passenger to receive it. Refreshing relevant pages or running process_expiries processes old offers and holds.

For a reliable class demo, use a future trip that is actually full and two passenger browsers. Do not book seats on a departed or cancelled trip. The preview_waitlist_demand management command can create a temporary nonempty dashboard example; press Enter in its PowerShell window to remove its probe data.

8. Account, reviews and customer help

Account allows a signed-in user to edit name, email and phone, change their password, and choose Light, Dark or device-based appearance. Profile fields can prefill a later booking; ticket details remain the saved booking snapshot. **Past journeys** lists booking history for trips whose arrival time has passed. Website login history is an extra staff-facing administration option, not the main account page.

After a **confirmed journey has arrived**, its passenger may leave **one 1–5 star review** on the booking page. A cancelled booking or a future journey is not eligible. Search results may show the average of verified reviews for a bus; an unrated bus says it has no verified reviews yet.

The site footer links to **Customer help / Support**. That page includes general FAQs about searching, seats, holds, charges, boarding, full trips, cancellations, tickets and reviews. A signed-in user can send a support request, optionally link one of their bookings, and track its status and any staff response on that page. Staff can inspect and answer requests in /admin/. This is an in-app demo help desk, **not** a live telephone helpline.

9. Staff pages and the DBMS story

Page	What staff can inspect
/admin/	Buses, physical seats, recurring templates, trips, points, users, bookings, payments, queue entries, reviews, support requests and audit events
/staff/overview/	Bus, trip, physical-place, trip-inventory, passenger, booking and cancellation totals; booking-state summary
/staff/revenue/	Tabular captured, refunded and net **simulated** amounts by trip
/insights/	Trip occupancy, simulated revenue and active waiting-list demand from Oracle reporting views

The database story is visible through **Oracle tables and relationships**, a separate trip-seat inventory, primary/foreign/candidate keys, functional dependencies and normalization, constraints, four project triggers, conditional unique indexes, atomic transactions, row locks, three reporting views, analytic FIFO position, execution plans and a Data Pump restore drill. For a professor who asks about ER diagrams, keys and FDs, open docs/er_model.md and docs/fd_normalization_worked_example.md. docs/course_alignment.md maps lecture topics to the implementation. docs/course_sql_lab.sql, docs/live_sql_demo.sql, docs/dbms_evidence.sql and docs/query_plans.sql provide SQL Developer exercises. The current detailed report is docs/BusBook_Detailed_Project_Report.pdf.

The optional verification commands include verify_integrity, verify_seat_race, verify_atomic_hold, verify_waitlist_fifo, verify_waitlist_leave, verify_active_waitlist, verify_booking_audit, verify_offer_rules, verify_trip_schedule, verify_fleet_layout and verify_bill_integrity. Use them from a PowerShell window with DB_PASSWORD set. Recorded results and their dates are in docs/observed_dbms_results.md; do not present older counts as live totals.

10. Three-person demonstration plan

Person	Suggested action	DBMS point to explain
Leader	Start host, show admin fleet/recurrence, SQL Developer ER/schema, constraints, triggers and report views	Relational architecture and database-owned rules
Teammate A	Filter a future trip, choose places/points, show bill, hold and confirm, print ticket	Inventory, atomic hold, locks, payment and bill constraint
Teammate B	Join a full trip's queue; watch a cancellation create an offer; accept and confirm it	FIFO analytic view, conditional uniqueness and timed offer

Before class, check Oracle and listener, open the site on the host, test the teammate link on another device on the **actual network**, and identify a future trip with the right availability. Prepare two separate passenger accounts. Keep the launcher window open, and have SQL Developer, the report and slides ready. Network changes require restarting the launcher. If a live queue sequence takes too long, explain the recorded verify_waitlist_fifo output rather than claiming a result the audience did not see.

11. Quick fixes and safe GitHub use

Symptom	First check
ORA-01017	Re-enter the Oracle BUSBOOKING password in this PowerShell window; check that the connection targets xepdb1.
ORA-12514	Oracle service may not have registered yet after restart; check lsnrctl services for xepdb1, then retry.
Listener fails with an old host name	Check listener.ora host setting and start the listener; do not change database files or user accounts just for this.
ORA-00942	Confirm SQL Developer uses BUSBOOKING in xepdb1 and migrations completed.
Teammate link fails	Confirm same network, current printed IP, host server, Private-network firewall rule; try a hotspot if campus Wi-Fi isolates devices.
Port 8000 already used	Stop the earlier Django server with Ctrl+C.
No trips found	Clear filters; choose a future date; check bus active status and generated trips.
Trip appears full unexpectedly	Refresh; live holds/offers may temporarily reserve places.
PowerShell blocks direct script	Use the powershell -NoProfile -ExecutionPolicy Bypass -File .\start_team.ps1 command.

GitHub is for versioned code, migrations, reports and manuals. It does not back up live Oracle data. The host may commit offline and push when connected. Before a checkpoint, inspect git status --short, stage only intended files, then commit and push. Never commit .env, database passwords, .venv, server-side Data Pump dumps or personal booking data. For installation detail, see docs/setup_and_network.md; for a short access guide, see docs/team_access.md.